

LicenseGuard

Software License Management System

DATABASE DESIGN, ENTITY-RELATIONSHIP DIAGRAM (ERD) & SQL IMPLEMENTATION

Project	LicenseGuard
Purpose	Software license inventory, assignment, expiry and renewal management
Database	MySQL
Primary Design	Relational database with 7 entities/tables
ER Model	Conceptual ERD using Chen-style entities, attributes and relationships

1. E-R Diagram (ERD)

An Entity-Relationship Diagram (ERD) is a visual representation of the LicenseGuard data model. It shows the entities, their attributes, the relationships between entities, and the cardinality of those relationships.

For LicenseGuard, each main entity corresponds to a database table. Primary keys identify records uniquely, while foreign keys connect related tables.

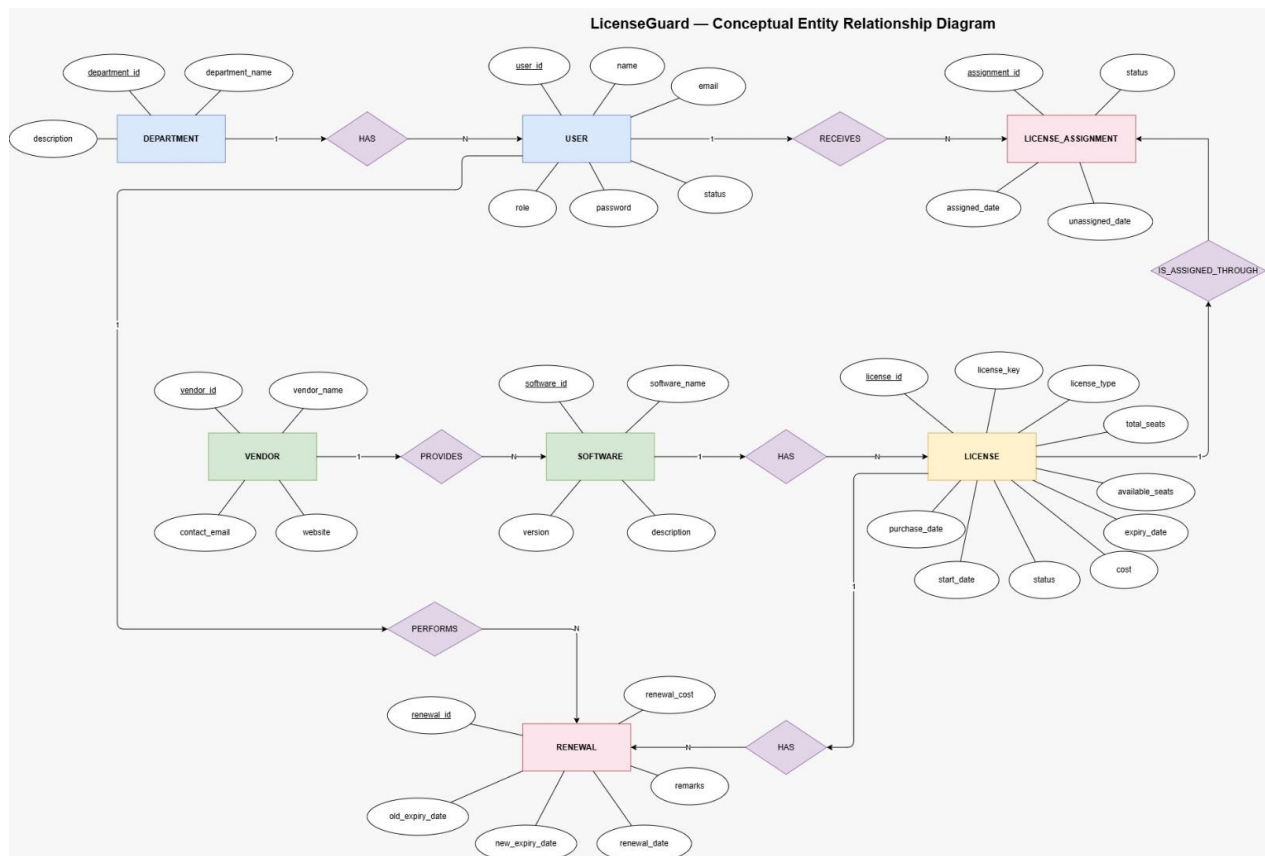


Figure 1: LicenseGuard Conceptual Entity-Relationship Diagram

2. Identify Entities

The main entities in the LicenseGuard software license management system are:

No.	Entity	Purpose
1	DEPARTMENT	Stores company departments.
2	USER	Stores employees/system users who may receive licenses.
3	VENDOR	Stores software vendors or publishers.
4	SOFTWARE	Stores software products and versions.
5	LICENSE	Stores purchased software license information.
6	LICENSE_ASSIGNMENT	Records which user receives which license.
7	RENEWAL	Stores license renewal history.

3. Define Attributes

The following attributes become columns in the corresponding MySQL tables. PK means Primary Key and FK means Foreign Key.

DEPARTMENT

Attribute	Data Type	Key / Constraint	Description
department_id	INT	PK, AUTO_INCREMENT	Unique department identifier.
department_name	VARCHAR(100)	NOT NULL	Name of the department.
description	VARCHAR(255)	Optional	Short department description.

```
mysql> DESC departments;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| department_id  | int           | NO   | PRI | NULL    | auto_increment |
| department_name | varchar(100)  | NO   |     | NULL    |                |
| description    | varchar(255)  | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

Figure 2: DEPARTMENT table structure screenshot

USER

Attribute	Data Type	Key / Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier.
name	VARCHAR(100)	NOT NULL	Full name of the user.
email	VARCHAR(150)	UNIQUE, NOT NULL	Login/contact email.
password	VARCHAR(255)	NOT NULL	Password value; production systems should store a secure hash.
role	VARCHAR(30)	NOT NULL	ADMIN, MANAGER or EMPLOYEE.
department_id	INT	FK	References DEPARTMENT.
status	VARCHAR(20)	DEFAULT ACTIVE	User status.

```
mysql> DESC users;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| user_id        | int           | NO   | PRI | NULL    | auto_increment |
| name           | varchar(100)  | NO   |     | NULL    |                |
| email          | varchar(150)  | NO   | UNI | NULL    |                |
| password       | varchar(255)  | NO   |     | NULL    |                |
| role           | varchar(30)   | NO   |     | NULL    |                |
| department_id  | int           | YES  | MUL | NULL    |                |
| status         | varchar(20)   | YES  |     | ACTIVE  |                |
+-----+-----+-----+-----+-----+-----+
```

Figure 3: USER table structure screenshot

VENDOR

Attribute	Data Type	Key / Constraint	Description
vendor_id	INT	PK, AUTO_INCREMENT	Unique vendor identifier.

Attribute	Data Type	Key / Constraint	Description
vendor_name	VARCHAR(100)	NOT NULL	Vendor/publisher name.
contact_email	VARCHAR(150)	Optional	Vendor contact email.
website	VARCHAR(255)	Optional	Vendor website.

```
mysql> DESC vendors;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| vendor_id  | int           | NO   | PRI | NULL    | auto_increment |
| vendor_name | varchar(100)  | NO   |     | NULL    |                |
| contact_email | varchar(150) | YES  |     | NULL    |                |
| website    | varchar(255)  | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

Figure 4: VENDOR table structure screenshot

SOFTWARE

Attribute	Data Type	Key / Constraint	Description
software_id	INT	PK, AUTO_INCREMENT	Unique software identifier.
software_name	VARCHAR(100)	NOT NULL	Software product name.
version	VARCHAR(50)	Optional	Software version.
vendor_id	INT	FK	References VENDOR.
description	TEXT	Optional	Software description.

```
mysql> DESC software;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| software_id | int           | NO   | PRI | NULL    | auto_increment |
| software_name | varchar(100)  | NO   |     | NULL    |                |
| version      | varchar(50)   | YES  |     | NULL    |                |
| vendor_id   | int           | YES  | MUL | NULL    |                |
| description  | text          | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

Figure 5: SOFTWARE table structure screenshot

LICENSE

Attribute	Data Type	Key / Constraint	Description
license_id	INT	PK, AUTO_INCREMENT	Unique license identifier.
software_id	INT	FK, NOT NULL	References SOFTWARE.
license_key	VARCHAR(255)	UNIQUE, NOT NULL	License key/identifier.
license_type	VARCHAR(50)	NOT NULL	Subscription, Perpetual, Enterprise, Volume, etc.
purchase_date	DATE	Optional	Date license was purchased.
start_date	DATE	Optional	License validity start date.
expiry_date	DATE	NOT NULL	License expiry date.
total_seats	INT	NOT NULL	Total purchased seats.

Attribute	Data Type	Key / Constraint	Description
available_seats	INT	NOT NULL	Currently unassigned seats.
cost	DECIMAL(10,2)	Optional	Purchase/license cost.
status	VARCHAR(30)	DEFAULT ACTIVE	ACTIVE, EXPIRING, EXPIRED or SUSPENDED.

```
mysql> DESC licenses;
```

Field	Type	Null	Key	Default	Extra
license_id	int	NO	PRI	NULL	auto_increment
software_id	int	NO	MUL	NULL	
license_key	varchar(255)	NO	UNI	NULL	
license_type	varchar(50)	NO		NULL	
purchase_date	date	YES		NULL	
start_date	date	YES		NULL	
expiry_date	date	NO		NULL	
total_seats	int	NO		NULL	
available_seats	int	NO		NULL	
cost	decimal(10,2)	YES		NULL	
status	varchar(30)	YES		ACTIVE	

Figure 6: LICENSE table structure screenshot

LICENSE_ASSIGNMENT

Attribute	Data Type	Key / Constraint	Description
assignment_id	INT	PK, AUTO_INCREMENT	Unique assignment identifier.
license_id	INT	FK, NOT NULL	References LICENSE.
user_id	INT	FK, NOT NULL	References USER.
assigned_date	DATE	NOT NULL	Date the license was assigned.
unassigned_date	DATE	Optional	Date the license was released.
status	VARCHAR(30)	DEFAULT ACTIVE	Assignment status.

```
mysql> DESC license_assignments;
```

Field	Type	Null	Key	Default	Extra
assignment_id	int	NO	PRI	NULL	auto_increment
license_id	int	NO	MUL	NULL	
user_id	int	NO	MUL	NULL	
assigned_date	date	NO		NULL	
unassigned_date	date	YES		NULL	
status	varchar(30)	YES		ACTIVE	

Figure 7: LICENSE_ASSIGNMENT table structure screenshot

RENEWAL

Attribute	Data Type	Key / Constraint	Description
renewal_id	INT	PK, AUTO_INCREMENT	Unique renewal identifier.

Attribute	Data Type	Key / Constraint	Description
license_id	INT	FK, NOT NULL	References LICENSE.
old_expiry_date	DATE	NOT NULL	Expiry date before renewal.
new_expiry_date	DATE	NOT NULL	Expiry date after renewal.
renewal_date	DATE	NOT NULL	Date renewal was performed.
renewal_cost	DECIMAL(10,2)	Optional	Cost of renewal.
renewed_by	INT	FK	References USER who performed renewal.
remarks	VARCHAR(255)	Optional	Additional renewal notes.

```
mysql> DESC renewals;
```

Field	Type	Null	Key	Default	Extra
renewal_id	int	NO	PRI	NULL	auto_increment
license_id	int	NO	MUL	NULL	
old_expiry_date	date	NO		NULL	
new_expiry_date	date	NO		NULL	
renewal_date	date	NO		NULL	
renewal_cost	decimal(10,2)	YES		NULL	
renewed_by	int	YES	MUL	NULL	
remarks	varchar(255)	YES		NULL	

Figure 8: RENEWAL table structure screenshot

4. Identify Relationships

LicenseGuard uses relationships between departments, users, vendors, software products, licenses, assignments and renewals. The following table describes the relationship and its cardinality.

Relationship	Cardinality	Meaning
DEPARTMENT — HAS — USER	1 : N	One department can contain many users; each user belongs to one department.
VENDOR — PROVIDES — SOFTWARE	1 : N	One vendor can provide many software products; each software product belongs to one vendor.
SOFTWARE — HAS — LICENSE	1 : N	One software product can have many license records.
USER — RECEIVES — LICENSE_ASSIGNMENT	1 : N	One user can have many license assignments.
LICENSE — IS ASSIGNED THROUGH — LICENSE_ASSIGNMENT	1 : N	One license can appear in many assignment records over time.
LICENSE — HAS — RENEWAL	1 : N	One license can have many renewal records.
USER — PERFORMS — RENEWAL	1 : N	One user/admin can perform many renewals.

5. Cardinality Notation

LicenseGuard mainly uses one-to-many (1:N) relationships. The LICENSE_ASSIGNMENT entity records license usage for users and allows assignment history to be maintained over time.

Notation	Meaning in LicenseGuard
1	Exactly one related record.
N	Many related records.
1:N	One parent record can be related to many child records.
0..N	Zero or many related records may exist.

6. Optional Attributes and Constraints

- Primary keys: uniquely identify each record.
- Foreign keys: maintain relationships between tables.
- UNIQUE: prevents duplicate user emails and license keys.
- NOT NULL: ensures required business information is present.
- AUTO_INCREMENT: automatically generates numeric identifiers.
- Status fields: support active/inactive or lifecycle tracking instead of unnecessary deletion.

7. Creating a Database

Using MySQL Server / MySQL Command Line Client, create a new database for the LicenseGuard system.

```
CREATE DATABASE licenseguard;
```

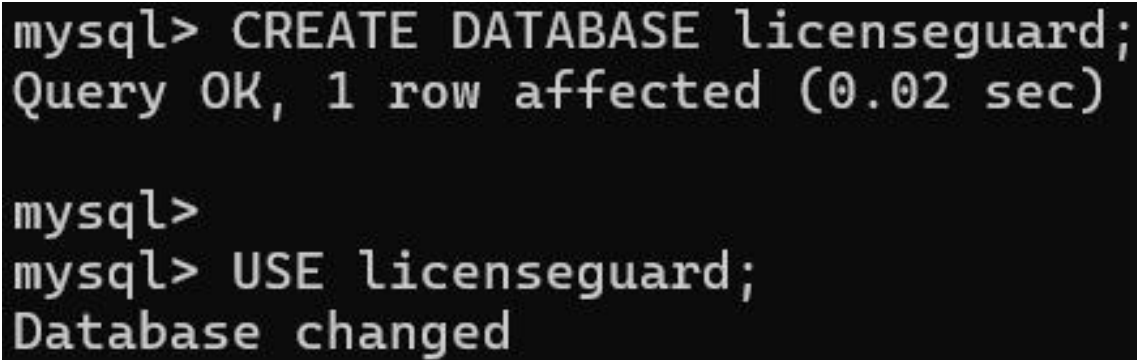
A screenshot of a MySQL command-line interface with a black background and white text. The first command is 'mysql> CREATE DATABASE licenseguard;', followed by the output 'Query OK, 1 row affected (0.02 sec)'. After a blank line, the prompt 'mysql>' is shown again, followed by the command 'mysql> USE licenseguard;' and the output 'Database changed'.

Figure 9: MySQL database creation and USE command output

8. Using the Database

Before performing operations on the tables, select the LicenseGuard database using the USE statement.

```
USE licenseguard;
```

The supplied MySQL screenshot in Figure 9 shows the database being selected successfully.

9. Creating the Tables for Each Entity

9.1 Create DEPARTMENT

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY AUTO_INCREMENT,  
    department_name VARCHAR(100) NOT NULL,  
    description VARCHAR(255)  
);
```

9.2 Create USER

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(30) NOT NULL,  
    department_id INT,  
    status VARCHAR(20) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_user_department  
        FOREIGN KEY (department_id)  
        REFERENCES departments(department_id)  
);
```

9.3 Create VENDOR

```
CREATE TABLE vendors (  
    vendor_id INT PRIMARY KEY AUTO_INCREMENT,  
    vendor_name VARCHAR(100) NOT NULL,  
    contact_email VARCHAR(150),  
    website VARCHAR(255)  
);
```

9.4 Create SOFTWARE

```
CREATE TABLE software (  
    software_id INT PRIMARY KEY AUTO_INCREMENT,  
    software_name VARCHAR(100) NOT NULL,  
    version VARCHAR(50),  
    vendor_id INT,  
    description TEXT,  
    CONSTRAINT fk_software_vendor  
        FOREIGN KEY (vendor_id)  
        REFERENCES vendors(vendor_id)  
);
```

9.5 Create LICENSE

```
CREATE TABLE licenses (  
    license_id INT PRIMARY KEY AUTO_INCREMENT,  
    software_id INT NOT NULL,  
    license_key VARCHAR(255) NOT NULL UNIQUE,  
    license_type VARCHAR(50) NOT NULL,  
    purchase_date DATE,  
    start_date DATE,  
    expiry_date DATE NOT NULL,  
    total_seats INT NOT NULL,  
    available_seats INT NOT NULL,  
    cost DECIMAL(10,2),  
    status VARCHAR(30) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_license_software  
        FOREIGN KEY (software_id)  
        REFERENCES software(software_id)  
);
```

9.6 Create LICENSE_ASSIGNMENT

```
CREATE TABLE license_assignments (  
    assignment_id INT PRIMARY KEY AUTO_INCREMENT,  
    license_id INT NOT NULL,  
    user_id INT NOT NULL,  
    assigned_date DATE NOT NULL,  
    unassigned_date DATE,  
    status VARCHAR(30) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_assignment_license  
        FOREIGN KEY (license_id)  
        REFERENCES licenses(license_id),  
    CONSTRAINT fk_assignment_user  
        FOREIGN KEY (user_id)  
        REFERENCES users(user_id)  
);
```

9.7 Create RENEWAL

```
CREATE TABLE renewals (  
    renewal_id INT PRIMARY KEY AUTO_INCREMENT,  
    license_id INT NOT NULL,  
    old_expiry_date DATE NOT NULL,  
    new_expiry_date DATE NOT NULL,  
    renewal_date DATE NOT NULL,  
    renewal_cost DECIMAL(10,2),  
    renewed_by INT,  
    remarks VARCHAR(255),  
    CONSTRAINT fk_renewal_license  
        FOREIGN KEY (license_id)  
        REFERENCES licenses(license_id),  
    CONSTRAINT fk_renewal_user  
        FOREIGN KEY (renewed_by)  
        REFERENCES users(user_id)  
);
```

10. Insert Records

Add sample records to the tables to test the LicenseGuard database. The records are inserted in dependency order so foreign-key constraints are satisfied.

10.1 Insert Departments

```
INSERT INTO departments (department_name, description) VALUES
('IT', 'Information Technology Department'),
('HR', 'Human Resources Department'),
('Finance', 'Finance and Accounts Department'),
('Administration', 'General Administration Department');
```

10.2 Insert Users

```
INSERT INTO users
(name, email, password, role, department_id, status) VALUES
('Admin User', 'admin@licenseguard.com', 'admin123', 'ADMIN', 1, 'ACTIVE'),
('Rahul Sharma', 'rahul@licenseguard.com', 'rahul123', 'EMPLOYEE', 1, 'ACTIVE'),
('Priya Patel', 'priya@licenseguard.com', 'priya123', 'EMPLOYEE', 2, 'ACTIVE'),
('Amit Kumar', 'amit@licenseguard.com', 'amit123', 'MANAGER', 3, 'ACTIVE');
```

10.3 Insert Vendors

```
INSERT INTO vendors
(vendor_name, contact_email, website) VALUES
('Microsoft', 'support@microsoft.com', 'https://www.microsoft.com'),
('Adobe', 'support@adobe.com', 'https://www.adobe.com'),
('JetBrains', 'support@jetbrains.com', 'https://www.jetbrains.com'),
('Autodesk', 'support@autodesk.com', 'https://www.autodesk.com');
```

10.4 Insert Software

```
INSERT INTO software
(software_name, version, vendor_id, description) VALUES
('Microsoft 365', '2026', 1, 'Productivity and office software suite'),
('Adobe Photoshop', '2026', 2, 'Image editing software'),
('IntelliJ IDEA', '2026', 3, 'Java development IDE'),
('AutoCAD', '2026', 4, 'Computer-aided design software');
```

10.5 Insert Licenses

```
INSERT INTO licenses
(software_id, license_key, license_type, purchase_date,
start_date, expiry_date, total_seats, available_seats,
cost, status) VALUES
(1, 'MS365-LG-001', 'Subscription', '2026-01-10',
'2026-01-10', '2027-01-09', 10, 8, 150000.00, 'ACTIVE'),
(2, 'ADOBE-LG-002', 'Subscription', '2026-02-15',
'2026-02-15', '2027-02-14', 5, 4, 90000.00, 'ACTIVE'),
(3, 'IJ-LG-003', 'Subscription', '2026-03-01',
'2026-03-01', '2027-02-28', 5, 3, 65000.00, 'ACTIVE'),
(4, 'ACAD-LG-004', 'Enterprise', '2026-04-01',
'2026-04-01', '2027-03-31', 3, 2, 120000.00, 'ACTIVE');
```

10.6 Insert License Assignments

```
INSERT INTO license_assignments
(license_id, user_id, assigned_date, unassigned_date, status) VALUES
(1, 2, '2026-01-15', NULL, 'ACTIVE'),
(1, 3, '2026-01-20', NULL, 'ACTIVE'),
(2, 2, '2026-02-20', NULL, 'ACTIVE'),
(3, 2, '2026-03-05', NULL, 'ACTIVE'),
(3, 4, '2026-03-06', NULL, 'ACTIVE'),
(4, 3, '2026-04-05', NULL, 'ACTIVE');
```

10.7 Insert Renewal

```
INSERT INTO renewals
(license_id, old_expiry_date, new_expiry_date,
renewal_date, renewal_cost, renewed_by, remarks) VALUES
(1, '2026-01-09', '2027-01-09',
'2026-01-05', 150000.00, 1,
'Annual Microsoft 365 renewal');
```

11. Select Records

SQL SELECT statements are used to retrieve and verify information stored in LicenseGuard.

11.1 Retrieve all departments

```
SELECT * FROM departments;
```

11.2 Retrieve all users

```
SELECT * FROM users;
```

11.3 Retrieve all vendors

```
SELECT * FROM vendors;
```

11.4 Retrieve all software

```
SELECT * FROM software;
```

11.5 Retrieve all licenses

```
SELECT * FROM licenses;
```

11.6 Find active licenses

```
SELECT license_id, license_key, license_type, expiry_date, status
FROM licenses
WHERE status = 'ACTIVE';
```

11.7 Find licenses expiring soon

```
SELECT license_id, license_key, expiry_date
FROM licenses
WHERE expiry_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY)
AND status <> 'EXPIRED';
```

11.8 View Users with Their Departments

```
SELECT u.user_id, u.name, u.email,
       d.department_name, u.role, u.status
FROM users u
LEFT JOIN departments d
      ON u.department_id = d.department_id;
```

11.9 View License Assignment Details

```
SELECT la.assignment_id,
       u.name AS user_name,
       s.software_name,
       l.license_key,
       la.assigned_date,
       la.status
FROM license_assignments la
JOIN users u ON la.user_id = u.user_id
JOIN licenses l ON la.license_id = l.license_id
JOIN software s ON l.software_id = s.software_id;
```

12. Update Records

UPDATE statements modify existing records. For LicenseGuard, status-based updates are preferred over physically deleting business records.

12.1 Update a User's Email

```
UPDATE users
SET email = 'rahul.sharma@licenseguard.com'
WHERE user_id = 2;
```

12.2 Update License Status

```
UPDATE licenses
SET status = 'SUSPENDED'
WHERE license_id = 4;
```

12.3 Mark an Assignment as Inactive

```
UPDATE license_assignments
SET status = 'INACTIVE',
    unassigned_date = CURDATE()
WHERE assignment_id = 1;
```

13. Delete Records

Ideally, important license-management records should not be physically deleted because they may be required for audit history. Use status fields such as ACTIVE, INACTIVE, EXPIRED or SUSPENDED whenever possible.

13.1 Soft Delete / Deactivate a User

```
UPDATE users
SET status = 'INACTIVE'
WHERE user_id = 3;
```

13.2 Physical Delete — Use Only When Appropriate

```
DELETE FROM users
WHERE user_id = 3;
```

Warning: This DELETE may fail if related records exist because foreign-key constraints protect referential integrity. In the actual LicenseGuard application, prefer deactivation or archiving rather than deleting historical records.

14. Database Verification

After creating the database and tables, use the following commands to verify the structure.

```
SHOW DATABASES;

USE licenseguard;

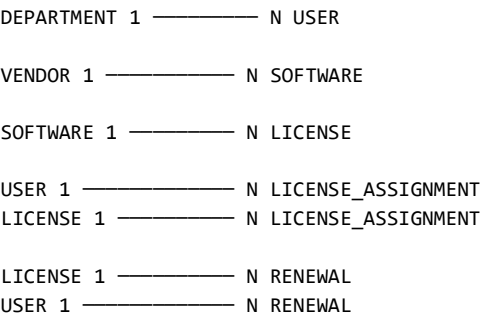
SHOW TABLES;

DESC departments;
DESC users;
DESC vendors;
DESC software;
DESC licenses;
DESC license_assignments;
DESC renewals;
```

15. Table Structure Summary

Table	Primary Key	Important Foreign Keys	Main Purpose
departments	department_id	—	Department master data
users	user_id	department_id	Application users/employees
vendors	vendor_id	—	Software vendors
software	software_id	vendor_id	Software products
licenses	license_id	software_id	Purchased licenses
license_assignments	assignment_id	license_id, user_id	License-to-user assignment history
renewals	renewal_id	license_id, renewed_by	Renewal history

16. LicenseGuard ERD Relationship Summary



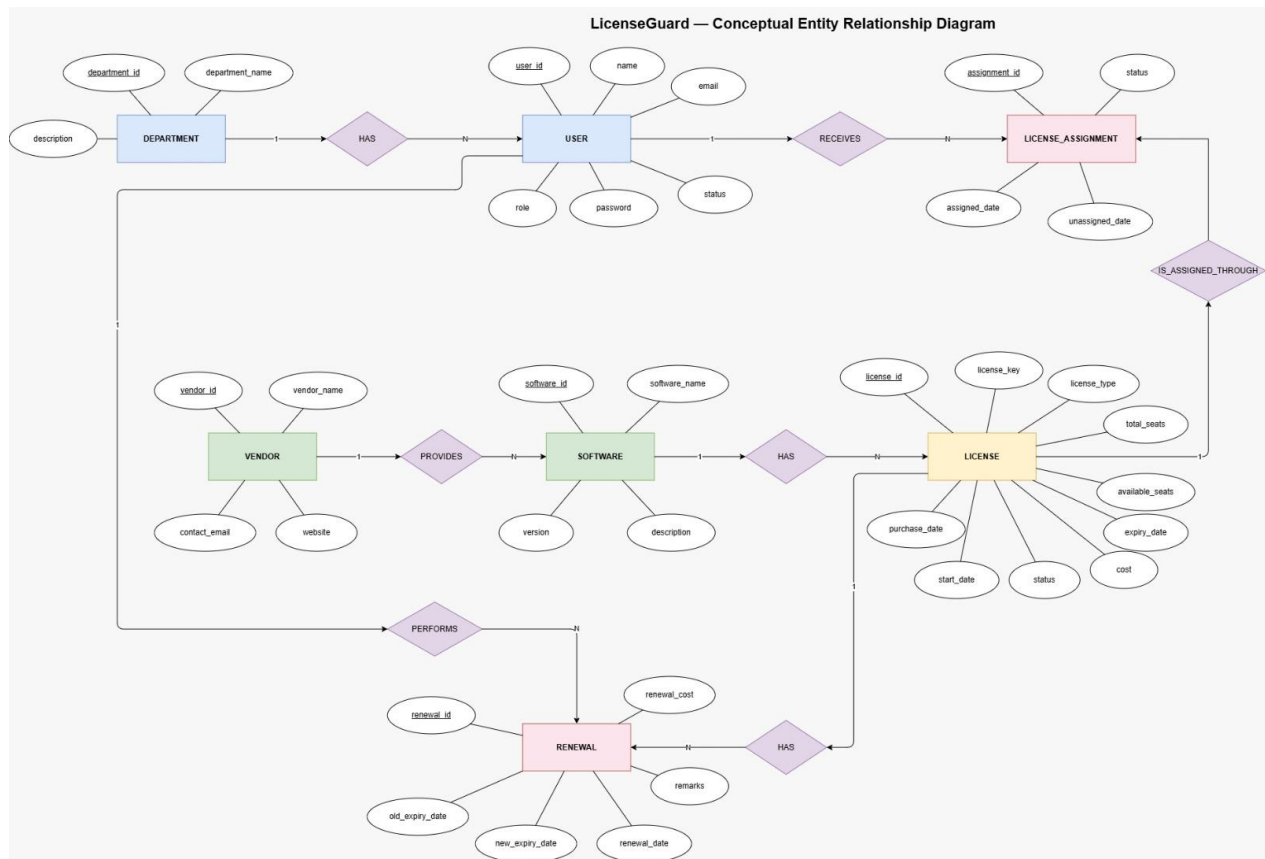


Figure 10: Final LicenseGuard ERD